Product + Project Management in Jira

Practical Operating System

Contents

Part I

Foundations

0.1 Jira as an Operating System: Work as a Graph

Jira is not a to-do list. It is an operating system for work where issues are nodes, links express dependencies and intent, statuses show state, and time creates the rhythm. This mental model lets you reason about delivery at scale: you can trace how a customer request becomes a release, see where work is stuck, and understand which commitments depend on which teams.

0.1.1 Mental model: issues, links, status, time

Think of every issue as a node with attributes (type, owner, priority, risk). Links define relationships: blocks, depends on, relates to, duplicates. Statuses represent the workflow state, not the work's value. Time introduces the cadence of decision-making through sprints, releases, and review windows. The system works when links are meaningful, statuses are consistent, and timeboxes are respected.

0.1.2 Worked scenario: a new feature request arrives from sales

A sales leader requests "bulk user import" for an enterprise deal. The PM creates an Epic with a clear outcome and links it to an Initiative. The PM captures a Story for the core capability and a Spike for technical unknowns. The TPM adds a dependency link to the data team. The EM assigns the Story to the delivery workflow and sets the target release. The graph now shows who owns the work, what must happen first, and when it is expected to land.

0.1.3 Case study insert

PulseBoard starts with Opportunity OPP-1 in Intake, which becomes Initiative INIT-1. Epic EPIC-1 is created for the checklist, and TS-104 is linked as a dependency so the graph makes the platform reliance explicit.

0.1.4 Checklist

- Every customer-facing request starts as an Epic linked to an Initiative or outcome.
- Links use explicit types: blocks, depends on, duplicates.
- Status represents workflow state, not progress estimates.
- Timeboxes (sprints/releases) are used consistently and reviewed on cadence.
- Dependencies are visible and owned, not hidden in comments.

0.1.5 Failure modes and fixes

1. **Failure:** Issues are unlinked, so dependencies are invisible. **Fix:** Require a link for any item with cross-team impact.
2. **Failure:** Statuses mean different things across teams. **Fix:** Define entry/exit criteria for each status and document it.
3. **Failure:** Timeboxes are created but never closed. **Fix:** Schedule a fixed sprint/release review and enforce closure.

4. **Failure:** Everything is a Story. **Fix:** Use Epics for outcomes and Tasks for non-user work.
5. **Failure:** Link types are vague (e.g., "relates to"). **Fix:** Default to blocks/depends on and audit weekly.

0.2 Roles and Responsibilities in Jira

Jira only works when ownership is explicit. PMs define outcomes and prioritize, POs keep the backlog clean and executable, TPMs manage dependencies and delivery risk, and EMs own execution health. Jira makes these responsibilities visible if each role owns specific artifacts.

0.2.1 Roles mapped to Jira artifacts

PM: owns Initiatives/Epics, outcome metrics, and prioritization fields.
PO: owns Story/Task readiness, acceptance criteria, and backlog hygiene.
TPM: owns dependencies, delivery risks, release coordination.
EM: owns workflow health, WIP, and team commitments.

0.2.2 Worked scenario: triaging a cross-team dependency

A Story is blocked by a data migration owned by another team. The TPM creates a dependency link and adds a Risk field update. The EM sets a WIP limit to prevent new work while blocked. The PM adjusts the Epic timeline and logs the impact on the Initiative. Jira now shows both the dependency and the business impact.

0.2.3 Case study insert

For OPP-2 and EPIC-5, the PM owns outcome fields, the PO keeps ST-115 and ST-116 in Ready, the TPM manages the dependency on TS-117, and the EM tracks WIP on the Monetization board.

0.2.4 Checklist

- PMs own outcome fields and Epic priority ordering.
- POs keep Stories in Ready with clear acceptance criteria.
- TPMs update dependency links and delivery risks weekly.
- EMs enforce WIP limits and update sprint commitments.
- Each artifact has a single accountable owner.

0.2.5 Failure modes and fixes

1. **Failure:** No one owns dependency links. **Fix:** Assign TPM ownership and review weekly.
2. **Failure:** PMs adjust scope but Jira stays stale. **Fix:** Require Epic updates before roadmap reviews.
3. **Failure:** POs accept Stories without criteria. **Fix:** Gate Ready status on acceptance fields.
4. **Failure:** EMs use Jira only after slips. **Fix:** Review WIP and flow weekly.
5. **Failure:** Multiple owners update the same field. **Fix:** Define field ownership and enforce it.

0.3 From Project Plans to Continuous Delivery

Slide plans describe intent; Jira describes execution. Move from static project plans to continuous delivery by turning roadmaps into a living backlog, linking work to releases, and reviewing progress through actual flow data instead of status decks.

0.3.1 How Jira replaces slide plans

A slide plan shows a timeline; Jira shows a network of commitments. Replace milestone slides with release versions and Epic progress. Replace weekly status decks with dashboards that show cycle time, progress to release, and risk flags. The result is less narrative and more evidence.

0.3.2 Worked scenario: quarterly plan update

A quarterly plan is due. Instead of a deck, the PM uses Jira releases and Epic progress to show what will ship. The TPM highlights blocked dependencies. The EM shows throughput and WIP to justify capacity. The review focuses on trade-offs rather than optimistic dates.

0.3.3 Case study insert

The quarterly plan for PulseBoard is expressed as Release R1 with EPIC-1, EPIC-4, and EPIC-5. Status slides are replaced by Jira release progress and the dependency list for TS-104 and TS-117.

0.3.4 Checklist

- Roadmap items are represented as Epics linked to releases.
- Release versions replace slide milestones.
- Dashboards replace status slides for weekly reporting.
- Capacity is grounded in throughput and WIP, not optimism.
- Risk and dependency fields are updated before review meetings.

0.3.5 Failure modes and fixes

1. **Failure:** Teams maintain both slides and Jira. **Fix:** Make Jira the source and export views for execs.
2. **Failure:** Releases are defined but never updated. **Fix:** Tie release updates to sprint review cadence.
3. **Failure:** Progress is reported as percent complete. **Fix:** Report based on done scope and flow metrics.
4. **Failure:** Risk is discussed but not recorded. **Fix:** Require Risk field updates for every Epic review.
5. **Failure:** Stakeholders demand fixed dates without evidence. **Fix:** Use throughput data and dependency links to reset expectations.

0.4 High-Leverage Habits: Triage, Refinement, Review Cadence

Jira only works if the team runs the system. High-leverage habits keep the backlog clean, the flow predictable, and the reporting credible. The goal is to make decisions quickly and consistently.

0.4.1 Cadences that keep the system healthy

Triage happens daily or twice weekly to prevent backlog rot. Refinement happens weekly to keep Ready items small and clear. Reviews happen on sprint or release cadence to keep commitments real. Each cadence updates fields that drive decisions: priority, risk, dependency, and target release.

0.4.2 Worked scenario: weekly refinement

The PO reviews the backlog and moves two items to Ready after adding acceptance criteria. The PM adjusts priority based on customer impact. The TPM flags a dependency and adds a mitigation note. The EM confirms capacity and pulls the top item into the sprint. Jira now reflects a credible plan.

0.4.3 Case study insert

During refinement, ST-102 and ST-116 move to Ready once acceptance criteria and Risk are filled. The PM reprioritizes EPIC-2 based on BG-122 frequency, and the update is visible in Jira.

0.4.4 Checklist

- Daily or twice-weekly triage clears new intake.
- Weekly refinement ensures Ready items meet DoR.
- Sprint/release reviews update Target Release and Risk.
- Backlog size stays within a 1–2 sprint horizon.
- Priority changes are logged and owned.

0.4.5 Failure modes and fixes

1. **Failure:** Backlog grows without pruning. **Fix:** Archive or close items that miss the next two cycles.
2. **Failure:** Refinement skips acceptance criteria. **Fix:** Enforce DoR before moving to Ready.
3. **Failure:** Triage becomes a status meeting. **Fix:** Limit triage to intake decisions only.
4. **Failure:** Reviews focus on activity, not outcomes. **Fix:** Tie review agenda to Epics and outcomes.
5. **Failure:** Priority changes are ad hoc. **Fix:** Require explicit customer impact and risk updates.

Part II

Delivery Operating System

0.5 Scrum in Jira: From Backlog to Sprint

Scrum in Jira is a cadence contract: the backlog is ready, the sprint has a goal, and scope changes are explicit. Use Jira to make the contract visible and enforceable.

0.5.1 Jira setup steps (Scrum)

1. Create a Scrum board scoped to your project (Board settings → Filter).
2. Define the Delivery workflow: Backlog → Ready → In Progress → In Review → Done.
3. Add fields to the screen: Customer Impact, Risk, Effort, Target Release.
4. Configure the board columns to match workflow states.
5. Enable sprint management and set the default sprint length.
6. Create a Definition of Ready checklist and require it before moving to Ready.

0.5.2 Backlog refinement

Refinement is where you remove ambiguity. The PO keeps items small, updates acceptance criteria, and confirms dependencies. The PM adjusts priority based on impact and risk. The EM confirms capacity so Ready means "ready to pull."

Sample JQL filters:

- Ready for refinement: `project = PMH AND status = Backlog ORDER BY priority DESC`
- Missing acceptance criteria: `project = PMH AND status = Backlog AND Äcceptance Criteriaïs EMPTY`
- High-risk backlog items: `project = PMH AND status = Backlog AND Risk = High`

0.5.3 Sprint planning

Planning turns Ready items into a committed sprint. The sprint goal is written first, then items are pulled until capacity is full. Scope changes require explicit trade-offs.

Sample JQL filters:

- Sprint candidates: `project = PMH AND status = Ready ORDER BY priority DESC`
- Unestimated in Ready: `project = PMH AND status = Ready AND Effort is EMPTY`

0.5.4 Sprint goal and scope control

Use the sprint goal to protect the sprint. If scope changes, update the goal and record what is traded out. Never add work without removing equivalent scope.

Sample JQL filters:

- Scope changes in sprint: `project = PMH AND sprint in openSprints() AND status = Backlog`
- Spillover from last sprint: `project = PMH AND sprint in closedSprints() AND status != Done`

0.5.5 Case study insert

Sprint 12 has a goal to deliver the setup checklist. The team pulls ST-101 and ST-102, and any change requires trading against EPIC-1 scope.

0.5.6 Cadence calendar

Table 1: Scrum cadence calendar

Cadence	Owner	Output
Backlog triage (2x/week)	PM/PO	Updated priority and intake decisions
Refinement (weekly)	PO/EM	Ready items with DoR met
Sprint planning (biweekly)	Team	Sprint goal and committed scope
Sprint review (biweekly)	PM/EM	Done scope and stakeholder feedback
Retro (biweekly)	EM	Improvement actions in Jira

0.6 Kanban in Jira: Flow and WIP

Kanban emphasizes flow over timeboxes. Jira can enforce flow discipline when WIP limits are explicit and aging work is visible.

0.6.1 Jira setup steps (Kanban)

1. Create a Kanban board scoped to your project or component.
2. Map workflow states to columns: Backlog, Ready, In Progress, In Review, Done.
3. Set WIP limits per column in Board settings.
4. Define a class of service field (Standard, Expedited, Fixed Date).
5. Add a swimlane by class of service.
6. Enable the Control Chart and Cumulative Flow Diagram.

0.6.2 WIP limits: representation and enforcement

WIP limits live in board column settings. Enforce them in daily standups: if a column is over limit, no new work enters it. Use automation or alerts if you need stricter enforcement.

Sample JQL filters:

- Over-limit columns (manual check): `project = PMH AND status = Ïn Progress¨`
- Expedited items: `project = PMH AND C̈lass of Service= Expedited AND status != Done`

0.6.3 Classes of service

Classes of service define priority rules. Expedited work preempts standard work but must be rare. Fixed Date work must include a target release or due date. Standard work follows normal flow.

Sample JQL filters:

- Fixed Date items: `project = PMH AND C̈lass of Service= F̈ixed Date¨`

0.6.4 Cycle time and aging work

Cycle time is the time from In Progress to Done. Aging work tracks items stuck too long. Use the Control Chart for cycle time and a JQL aging filter for stuck items.

Sample JQL filters:

- Aging work (In Progress > 7 days): `project = PMH AND status = "In Progress" AND updated <= -7d`
- Review bottleneck: `project = PMH AND status = "In Review" AND updated <= -3d`

0.6.5 Case study insert

The platform team runs Kanban. TS-107 and TS-117 stay within WIP limits, and any expedited item must be labeled and reviewed daily.

0.7 Bugs and Incidents: Fast Lanes and Learning

Bugs and incidents need speed and structure. Treat them as a separate flow with severity rules, fast lanes, and a postmortem pattern.

0.7.1 Jira setup steps (Bugs/Incidents)

1. Create a Bug issue type and add Severity (S1–S4) as a field or label.
2. Configure a Bug workflow: Triage → Fix In Progress → Verify → Done, with Won't Fix.
3. Create a fast-lane swimlane for Severity S1/S2.
4. Add a postmortem issue type or Task template linked to the Bug.
5. Automate creation of a postmortem Task when Severity is S1.

0.7.2 Severity model

S1: customer outage or data loss. S2: major degradation. S3: minor defect. S4: cosmetic. Severity determines SLA and escalation path.

Sample JQL filters:

- Active S1/S2: `project = PMH AND issuetype = Bug AND Severity in (S1, S2) AND status != Done`
- Bugs past SLA: `project = PMH AND issuetype = Bug AND status != Done AND updated <= -2d`

0.7.3 Fast lanes

Fast lanes are swimlanes or quick filters that surface critical issues. For S1/S2, bypass standard WIP limits but require EM or TPM approval to enter.

0.7.4 Postmortem issue pattern

For every S1, create a linked Task called "Postmortem: <incident>" with owners and due dates. Track root cause, fixes, and follow-up items.

Sample JQL filters:

- Missing postmortem: `project = PMH AND issuetype = Bug AND Severity = S1 AND issueLinkType is EMPTY`

0.7.5 Case study insert

BG-124 is marked S1 due to failed upgrade emails. The bug moves through the fast lane and spawns a postmortem task linked to EPIC-5.

0.8 Estimation and Forecasting

Estimation is a tool for planning, not a promise. Use points to compare relative size, and use throughput for forecasting. Keep the system honest by measuring what actually finishes.

0.8.1 Story points vs throughput

Story points help with sprint sizing and team learning. Throughput (items completed per sprint) is the strongest predictor of near-term capacity. Use points for in-sprint planning and throughput for external commitments.

Sample JQL filters:

- Completed last sprint: `project = PMH AND sprint in closedSprints() AND status = Done`
- Unestimated in current sprint: `project = PMH AND sprint in openSprints() AND Effort is EMPTY`

0.8.2 Probabilistic forecasting (no heavy math)

Use past throughput to create ranges. Example: if the team finishes 8–12 items per sprint, forecast delivery in 2–3 sprints for a 20-item Epic. The goal is a confidence range, not a single date. Update the range each sprint based on real throughput.

0.8.3 Case study insert

Estimates for ST-105 through ST-108 are used to size EPIC-2. Throughput from the last three sprints informs how much import work can fit in R1.

0.8.4 Release burndown pitfalls

Burndowns lie when scope changes. If scope grows, the burndown looks worse even if delivery is steady. Always show scope change alongside burn rate.

Sample JQL filters:

- Scope added mid-sprint: `project = PMH AND sprint in openSprints() AND createdDate >= startOfSprint()`

0.9 Forecasting with Flow Metrics

Forecasting improves when you use cycle time and WIP. Flow metrics show whether the system can absorb new work without breaking.

0.9.1 Jira setup steps (forecasting)

1. Enable Control Chart and Cumulative Flow Diagram on the board.
2. Define cycle time boundaries (In Progress to Done).
3. Set a target WIP level and monitor WIP breaches.
4. Build a dashboard widget for average cycle time and throughput.

Sample JQL filters:

- Done last 30 days: `project = PMH AND status = Done AND resolved >= -30d`
- WIP breach candidates: `project = PMH AND status in (Ïn Progress, Ïn Review)̈`

0.9.2 Worked example: forecasting a release window

An Epic contains 24 Stories. The team last six sprints show 8–10 Stories finished per sprint. The PM forecasts a 3-sprint window with a 70% confidence range, communicated as "3–4 sprints." Jira dashboards show cycle time and WIP; if WIP rises, the forecast widens.

0.9.3 Case study insert

Forecasting uses the completion rate of EPIC-4 and EPIC-5 stories. If cycle time rises, the R1 window is widened and the dashboard records the new range.

0.9.4 Release burndown pitfalls revisited

Use burndowns only for short horizons and stable scope. If scope shifts weekly, switch to throughput and cycle time to avoid false precision.

Part III

Product Discovery and Outcomes

0.10 Intake and Triage: From Requests to Opportunities

Raw requests arrive from sales, support, and leadership. Intake turns noise into structured opportunities. Triage decides what is worth discovery and what is not.

0.10.1 Jira setup steps (intake)

1. Create an intake queue (filter + board) scoped to new requests.
2. Use a dedicated issue type: Opportunity (preferred) or Epic with label "opportunity".
3. Add fields: Customer Impact, Reach, Confidence, Risk, OKR Link.
4. Add a lightweight triage workflow: Intake → Triage → Discovery → Parked → Closed.
5. Create automation: if label = "sales" then notify PM; if Risk = High then notify TPM.

0.10.2 Turning requests into opportunities

An opportunity is a structured hypothesis about customer value. It captures the problem, expected outcome, and why now. Raw requests should never go straight to delivery without a brief.

0.10.3 Worked scenario: support escalation becomes an opportunity

Support reports churn due to slow onboarding. The PM creates an Opportunity issue, links it to the churn OKR, and sets Customer Impact to High. The PO adds a hypothesis: "reducing setup time from 30 to 10 minutes will reduce churn by 5%." The item moves to Discovery with a clear owner and scope.

0.10.4 Case study insert

OPP-1 enters Intake from support data and is normalized into an Opportunity brief before it becomes INIT-1. The triage decision is recorded before any delivery work starts.

0.10.5 Checklist

- Every request is normalized into an Opportunity or Epic with an opportunity label.
- Customer Impact, Reach, and Risk are set before Discovery.
- No request jumps directly to Ready without a brief.
- Parked items have explicit review dates.
- Closed items include a reason and evidence.

0.10.6 Failure modes and fixes

1. **Failure:** Intake is a dumping ground. **Fix:** Enforce triage within 7 days.
2. **Failure:** Opportunities are vague. **Fix:** Require a hypothesis and outcome metric.
3. **Failure:** Everything becomes a top priority. **Fix:** Use explicit scoring and tie-breakers.

4. **Failure:** Parked items never return. **Fix:** Add review dates and automated reminders.
5. **Failure:** Requests bypass the system. **Fix:** Block delivery without an Opportunity link.

0.11 Discovery in Jira: Hypotheses, Assumptions, Experiments

Discovery turns opportunities into decisions. Jira can track the evidence without bloating the delivery backlog if you keep discovery issues lightweight and time-boxed.

0.11.1 Jira setup steps (discovery)

1. Add an Opportunity issue type or use Epics as opportunities.
2. Create a Discovery workflow: Intake → Discovery → Validated → Planned → Done.
3. Create an Experiment issue type (or Task subtype) linked to the Opportunity.
4. Add fields: Hypothesis, Assumptions, Evidence, Confidence.
5. Create a board or filter for Discovery items only.

0.11.2 Hypotheses and assumptions

Hypotheses describe expected outcomes. Assumptions describe what must be true for the outcome to hold. Keep both short and testable. Store them directly on the Opportunity or Experiment issue.

0.11.3 Experiment tasks

Each experiment is a time-boxed task with a clear success signal. Experiments are not features. They exist to reduce uncertainty.

0.11.4 Worked scenario: pricing hypothesis

The PM creates an Opportunity: "increase conversion on the Pro plan." Assumptions include "teams will pay for audit trails." An Experiment task is created to run a pricing test for two weeks. The outcome is recorded in Evidence and Confidence is updated before moving to Validated.

0.11.5 Case study insert

EXP-1 tests a shorter setup wizard for OPP-1. Evidence is captured in the Opportunity so INIT-1 can move to Validated.

0.11.6 Checklist

- Every Opportunity has a hypothesis and success metric.
- Assumptions are explicit and limited to 3–5 items.
- Experiments are time-boxed and linked to the Opportunity.
- Confidence is updated after evidence is collected.
- Validated means evidence exists, not just stakeholder agreement.

0.11.7 Failure modes and fixes

1. **Failure:** Discovery tasks linger indefinitely. **Fix:** Time-box experiments and close stale ones.
2. **Failure:** Hypotheses are too vague. **Fix:** Require a measurable outcome.
3. **Failure:** Evidence is scattered in docs. **Fix:** Store the summary in Jira fields.
4. **Failure:** Experiments become delivery work. **Fix:** Enforce size limits and separate issue type.
5. **Failure:** Confidence never changes. **Fix:** Update Confidence as a step in Discovery review.

0.12 Prioritization in Jira: RICE, WSJF, and Tie-Breakers

Prioritization frameworks keep debate focused on evidence. Jira can encode RICE or WSJF using simple fields so scoring is transparent and repeatable.

0.12.1 Jira setup steps (prioritization)

1. Add fields: Reach (number), Impact (single select), Confidence (single select), Effort (number).
2. Create a custom field for RICE Score (number) or calculate externally and paste.
3. Optional: add WSJF fields (Cost of Delay, Job Size).
4. Build a dashboard gadget that sorts by score.
5. Define tie-breaker rules in the project README or Confluence page.

0.12.2 RICE implementation

RICE = $(Reach \times Impact \times Confidence)/Effort$. Use a fixed scale for Impact and Confidence (e.g., 0.5, 1, 2). Keep scales consistent across teams.

0.12.3 WSJF implementation (optional)

WSJF = Cost of Delay / Job Size. Use it for portfolio-level trade-offs or large bets where delay cost is measurable.

0.12.4 Tie-breakers when scores are close

When scores are within 10%, use a tie-breaker list: risk reduction first, dependency unblock second, then time sensitivity. Document the decision in the Opportunity or Epic.

0.12.5 Worked scenario: two similar opportunities

Two Opportunities score 18 and 17 on RICE. The higher-risk item unlocks a partner integration. The PM applies the tie-breaker: dependency unblock wins. The decision is recorded and visible.

0.12.6 Case study insert

OPP-2 scores slightly lower than OPP-1 on RICE, but the tie-breaker favors dependency unblock on EPIC-5 because TS-117 is on the critical path.

0.12.7 Checklist

- Scoring fields are present and required before Ready.
- Impact and Confidence use a shared scale.
- Scores are recalculated when assumptions change.
- Tie-breaker rules are documented and used.
- Decisions are recorded on the issue.

0.12.8 Failure modes and fixes

1. **Failure:** Scores are inflated to win priority. **Fix:** Require evidence links for high scores.
2. **Failure:** Multiple scoring systems run in parallel. **Fix:** Pick one per product area.
3. **Failure:** Teams ignore tie-breakers. **Fix:** Make tie-breakers part of review cadence.
4. **Failure:** Effort is missing or inconsistent. **Fix:** Standardize estimation units.
5. **Failure:** Scores never change post-discovery. **Fix:** Re-score after new evidence.

0.13 Roadmaps and Commitments: Outcomes vs Features

A roadmap is a decision tool, not a contract. Outcome roadmaps track the change you want in the world; feature roadmaps track deliverables. You need both, but never confuse them.

0.13.1 Jira setup steps (roadmaps)

1. Use Initiatives (Advanced Roadmaps) for outcomes; Epics for features.
2. Link Epics to Initiatives with “implements” or parent-child.
3. Use Target Release for feature commitments; use OKR Link for outcomes.
4. Create two views: Outcome Roadmap (Initiatives) and Feature Roadmap (Epics).

0.13.2 How to avoid roadmap as contract

Use time ranges and confidence, not exact dates. Track dependencies and risk for every committed Epic. Communicate changes as trade-offs, not surprises.

0.13.3 Worked scenario: roadmap negotiation

An exec asks for a specific feature by end of quarter. The PM shows the Outcome Roadmap and explains the trade-off: delivering the feature will delay a retention outcome. The decision is recorded in the Epic with updated Target Release and Risk.

0.13.4 Case study insert

INIT-2 is the outcome roadmap item, while EPIC-4 and EPIC-5 are feature commitments. The team communicates R1 as a range, not a fixed promise.

0.13.5 Checklist

- Initiatives represent outcomes; Epics represent deliverables.
- Commitments include confidence level and dependencies.
- Outcome roadmap is reviewed monthly, feature roadmap biweekly.
- Changes are logged as trade-offs.
- Roadmap communication uses ranges, not fixed dates.

0.13.6 Failure modes and fixes

1. **Failure:** Roadmap is treated as a promise. **Fix:** Use ranges and confidence levels.
2. **Failure:** Outcomes are tracked as features. **Fix:** Separate Initiatives from Epics.
3. **Failure:** Dependencies are hidden. **Fix:** Require dependency links for commitments.
4. **Failure:** Reviews are ad hoc. **Fix:** Set fixed roadmap review cadence.
5. **Failure:** Changes are not recorded. **Fix:** Document trade-offs in the issue.

0.14 OKRs and Outcome Tracking

OKRs connect delivery to measurable change. In Jira, OKRs should be linked to Initiatives and Epics so outcomes are visible in the same system as execution.

0.14.1 Jira setup steps (OKRs)

1. Add an OKR Link field (URL) to Initiatives and Epics.
2. Create a dashboard for OKR-linked work using JQL.
3. Add a measurable signal field (Metric Name, Baseline, Target).
4. Define a review workflow for OKR status (On Track, At Risk, Off Track).

0.14.2 Outcome tracking with measurable signals

Each Initiative should have one primary metric and one guardrail. Store the metric and baseline in the Initiative. Update progress on a cadence independent of sprint reviews.

0.14.3 Worked scenario: activation OKR

An OKR targets a 15% increase in activation. The PM links the Initiative to the OKR, sets the baseline, and creates Epics for onboarding improvements. After release, the metric is updated in the Initiative and the OKR status is set to On Track.

0.14.4 Case study insert

INIT-1 is linked to OKR-A with a baseline activation metric. Progress updates are recorded monthly after EPIC-1 changes ship.

0.14.5 Checklist

- Every Initiative has a linked OKR and metric.
- Baseline and target are recorded in Jira.
- Outcome reviews happen monthly, not only at sprint reviews.
- Epics roll up to Initiatives with clear ownership.
- Status updates include evidence (dashboards or reports).

0.14.6 Failure modes and fixes

1. **Failure:** OKRs exist outside Jira with no link. **Fix:** Require OKR Link for Initiatives.
2. **Failure:** Metrics are vague. **Fix:** Record baseline, target, and timeframe.
3. **Failure:** Reviews happen only at quarter end. **Fix:** Set monthly OKR review cadence.
4. **Failure:** Epics are not tied to outcomes. **Fix:** Enforce parent Initiative links.
5. **Failure:** Status is updated without evidence. **Fix:** Require a dashboard or report link.

Sample JQL filters:

- OKR-linked initiatives: `project = PMH AND issuetype = Initiative AND "OKR Link" is not EMPTY`
- Missing metrics: `project = PMH AND issuetype = Initiative AND "Metric Name" is EMPTY`

Part IV

Governance and Scale

0.15 Portfolio Structure and Dependencies

Scaling requires a portfolio structure that preserves clarity. Initiatives represent outcomes, Epics represent deliverables, and teams own execution. Use links for dependencies rather than shadow plans.

0.15.1 Portfolio structure

Initiatives roll up outcomes and span multiple Epics. Epics map to a team or a release. Stories and Tasks live in team backlogs. Teams stay accountable for delivery; the portfolio view stays accountable for outcomes.

0.15.2 Dependency tracking via issue links

Dependencies are explicit links. Use "blocks" and "depends on". Avoid vague relationships. Assign an owner to every dependency and review them on a cadence.

Sample JQL filters:

- Blocked items: `project = PMH AND issueLinkType = blocks AND status != Done`
- Dependencies by team: `project = PMH AND component = "Data" AND issueLinkType is not EMPTY`

0.15.3 Worked scenario: multi-team initiative

An Initiative requires changes from three teams. The PM links all Epics to the Initiative and adds dependency links between Epics. The TPM assigns owners to each dependency and reviews the critical path weekly. Jira now shows the true delivery path.

0.15.4 Case study insert

The portfolio view shows INIT-1 and INIT-2 spanning Growth and Monetization squads, with EPIC-2 and EPIC-5 linked as cross-team dependencies.

0.15.5 Failure modes and fixes

1. **Failure:** Initiatives are just large Epics. **Fix:** Define outcomes and assign measurable metrics.
2. **Failure:** Dependencies live in slides. **Fix:** Require Jira issue links for cross-team work.
3. **Failure:** Ownership is unclear. **Fix:** Assign a dependency owner and review weekly.
4. **Failure:** Teams create duplicate Epics. **Fix:** Centralize Initiative ownership and link Epics.
5. **Failure:** Portfolio views hide delivery risk. **Fix:** Require Risk and Target Release fields on Epics.

0.16 Governance Cadence: Reviews Without Theatre

Governance exists to make decisions, not to generate status theatre. Keep the cadence predictable and evidence-based.

0.16.1 Governance cadence

- Weekly portfolio review: top risks, blocked dependencies, key metrics. - Monthly product review: outcome progress, OKR status, learnings. - Quarterly planning: reset priorities, capacity, and major bets.

0.16.2 Worked scenario: weekly portfolio review

The portfolio review begins with blocked dependencies and critical risks. The TPM updates dependency owners. The PM adjusts Initiative status with evidence. The EM highlights capacity constraints. Decisions are logged as changes in Jira, not as notes in a deck.

0.16.3 Case study insert

In the weekly portfolio review, TS-117 is flagged as a blocker for EPIC-5. The TPM assigns an owner and the Jira link is updated during the meeting.

0.16.4 Failure modes and fixes

1. **Failure:** Reviews become status recitals. **Fix:** Require decisions or escalations in every review.
2. **Failure:** Metrics are too broad to act on. **Fix:** Focus on 3–5 metrics tied to outcomes.
3. **Failure:** Quarterly plans ignore execution data. **Fix:** Bring throughput and cycle time into planning.
4. **Failure:** Reviews are cancelled when busy. **Fix:** Treat reviews as the decision forum, not optional.
5. **Failure:** Jira is updated after the meeting. **Fix:** Update issues during the review.

0.17 Metrics That Matter: Outcomes and Delivery

Metrics should drive decisions. Keep a clean split between product outcomes and delivery health.

0.17.1 Delivery metrics

Lead time, cycle time, throughput, WIP, predictability, escaped defects. These measure execution health and should be reviewed weekly.

0.17.2 Product outcome metrics

Activation, retention, revenue, NPS, or other product-specific signals. These measure impact and should be reviewed monthly or quarterly.

Sample JQL filters:

- Done last 30 days: `project = PMH AND status = Done AND resolved >= -30d`
- Escaped defects (post-release bugs): `project = PMH AND issuetype = Bug AND created >= -30d AND "Target Release" is EMPTY`

0.17.3 Worked scenario: predictability check

The team throughput drops by 30% and WIP increases. The EM pauses new intake and rebalances work. The PM adjusts the release window. The portfolio view updates with a new forecast based on real flow metrics.

0.17.4 Case study insert

Escaped defect BG-122 is tracked as a post-release bug tied to EPIC-2. The metric review triggers a WIP pause until the defect rate drops.

0.17.5 Failure modes and fixes

1. **Failure:** Outcomes and delivery metrics are mixed. **Fix:** Separate dashboards and review cadences.
2. **Failure:** Metrics are collected but not used. **Fix:** Tie each metric to a decision rule.
3. **Failure:** Teams game the metrics. **Fix:** Use multiple indicators, not one KPI.
4. **Failure:** Escaped defects are ignored. **Fix:** Track post-release bugs and review monthly.
5. **Failure:** WIP grows unchecked. **Fix:** Enforce WIP limits and pause intake.

0.18 Anti-patterns at Scale

Scaling Jira without discipline leads to bureaucracy. Avoid the patterns that create noise without decisions.

0.18.1 Anti-patterns and fixes

1. **Status by comment.** Fix: require status updates in fields, not in comments.
2. **Field explosion.** Fix: every new field needs an owner and a usage rule.
3. **Boards per person.** Fix: build boards per team or workflow, not per individual.
4. **Everything is urgent.** Fix: define classes of service and enforce WIP limits.
5. **Governance theatre.** Fix: meetings must produce decisions or escalations.

0.18.2 Case study insert

When every item in PulseBoard was labeled urgent, the team introduced classes of service and removed personal boards to restore flow discipline.

Governance Operating Manual

Governance Operating Manual

Purpose

This manual defines the minimum governance loop for a product portfolio using Jira. The goal is to make decisions fast without creating theatre.

Cadence

- Weekly portfolio review (30–45 min)
- Monthly product review (60–90 min)
- Quarterly planning (half-day)

Weekly portfolio review

Inputs

- Top 10 Initiatives by risk or impact
- Blocked dependencies
- Critical path items
- Delivery metrics (cycle time, WIP, throughput)

Decisions

- Escalate or resolve blocked dependencies
- Adjust target releases when capacity shifts
- Reprioritize when risk exceeds tolerance

Outputs (record in Jira)

- Updated Initiative status
- Updated dependency owners
- New or removed risks

Monthly product review

Inputs

- Outcome metrics by Initiative
- OKR status and evidence
- Release outcomes and learnings

Decisions

- Continue, pivot, or stop initiatives
- Allocate or remove capacity
- Approve next experiments

Outputs (record in Jira)

- OKR status updates
- New experiments and owners
- Updated confidence and impact fields

Quarterly planning

Inputs

- Portfolio capacity and throughput
- Strategic goals and constraints
- Major dependencies and risks

Decisions

- Select top initiatives and funding levels
- Confirm release windows and delivery horizons
- Set guardrails for scope changes

Outputs (record in Jira)

- Initiative priority order
- Target releases
- Dependency map

Escalation rules

- Any S1 incident or risk of data loss escalates immediately.
- Any dependency blocked > 7 days goes to weekly review.
- Any Initiative off track for 2 consecutive reviews triggers a reset plan.

Anti-theatre rules

- No status-only updates. Every update must include a decision or a metric change.
- No new fields without a usage rule and owner.
- No duplicate roadmaps; Jira is the source.